# eMBB / URLLC Coexistence MAC Scheduler

## Open6G AI School — Group 6

**Platform:** OpenAirInterface NR gNB · Foxconn O-RU · 2×2 MIMO

**Language:** Pure Lua (LuaJIT FFI) — runs inside the gNB slot loop

# The Problem

5G NR must serve **two fundamentally different traffic classes** on the same carrier:

| | eMBB | URLLC |
|---|---|---|
| **Goal** | Maximize throughput | Minimize latency + ultra-reliability |
| **BLER target** | 10% | 0.1–1% |
| **Latency** | Best-effort | < 50 ms RTT |
| **Packet size** | Large (video, file) | Small (control, sensor) |
| **5QI** | 9 | 69 |

**Challenge:** URLLC needs *immediate* radio resources → must preempt eMBB without destroying throughput.

## Scheduler Architecture — 3 Phases per Slot

```
┌──────────────────────────────────────┐
│  compute_dl_allocations()            │
│  Called from C every DL slot         │
│  (~0.5 ms at 30 kHz SCS)             │
└──────────────────────────────────────┘
                  │
        ┌─────────┴─────────┐
        ▼                   ▼                   ▼
Phase 1: HARQ      Phase 2: URLLC     Phase 3: eMBB
Retransmissions     New Data            New Data
(highest priority)  (strict deadline)   (proportional fair)
```

Each phase consumes RBs from a shared **RB mask** — what's left flows to the next phase.

## Phase 2 — URLLC Scheduling

**Priority:** Sort by head-of-line delay (most urgent first)

**Frequency-selective:** Pick RBs with best channel magnitude per UE
→ Better reliability without wasting bandwidth

**Budget cap:** URLLC limited to 15–30% of free RBs per slot
→ Prevents URLLC from starving eMBB under bursty arrivals

**Deadline monitoring:** Log warnings when HOL > 2 ms

```
table.sort(urllc_ues, function(a, b) return a.hol > b.hol end)

local s = find_best_channel_rbs(mask, needed, bwp_start, bwp_size, channel_mag)
```

## Phase 3 — eMBB with Puncturing Compensation

Standard proportional fair: $\mathrm{coef}_i = \frac{r_i(t)}{T_i(t)}$

**Problem:** URLLC "punctures" eMBB RBs → some UEs lose throughput unfairly.

**Fix** (Karimi et al.): Track per-UE puncturing fraction $\rho_i$ via EMA:

$$\mathrm{coef}_i = \frac{r_i(t)}{T_i(t) \cdot (1 - \rho_i)}$$

UEs that lose more RBs to URLLC get **priority boost** in subsequent slots.

Each eMBB UE gets the **largest contiguous free block** → maximizes transport block size.

# OLLA — Outer Loop Link Adaptation

Per-RNTI fractional MCS tracking with **separate targets per class:**

| Parameter | eMBB | URLLC |
|---|---|---|
| BLER target | 10% | 1% |
| Step up | +0.50 | +0.50 |
| Step down | −0.50 | −0.50 |
| Init MCS | 14 | 9 |

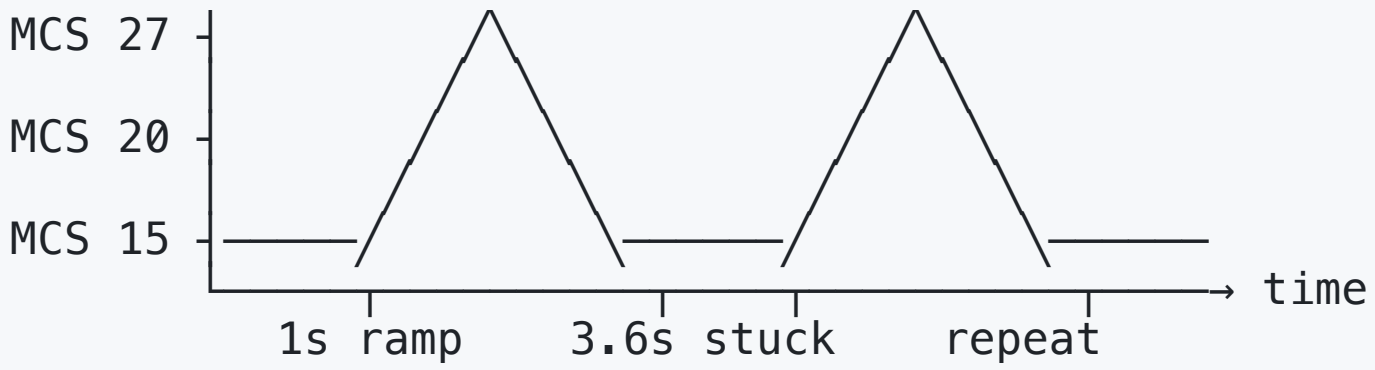
**CQI floor:** Prevents death spiral — MCS never drops below $\mathrm{CQI_MCS} - 6$

**Acceleration:** Ramps faster when MCS is far below CQI suggestion (1.25–1.5×)

**Taper:** Slows down within 3 MCS of ceiling to avoid overshoot

# The OLLA Oscillation Problem (v1–v3)

MCS exhibited a destructive **sawtooth** pattern:




**Root cause:** No memory that MCS 27 was unsustainable.

After crashing, OLLA climbs right back to the same failing MCS.

**Result:** ~96 Mbps avg with 2.2% loss, 3.6s throughput stalls.

## v4 Fix — Adaptive MCS Ceiling Tracking

**Key idea:** Remember where BLER spiked, don't go back there.

1. **Record ceiling** when BLER > 2× target → `mcs_ceiling = current_frac`
2. **Cap ramp-ups** at `ceiling - margin` (never reach the spike point again)
3. **Relax ceiling** slowly (+0.5/period) when BLER is good → cautious re-exploration
4. **BLER dead-zone** (1.3×): Hold MCS steady for minor BLER fluctuations
5. **Cumulative drop cap** (max −6 MCS per event): Prevents cascade to floor

```
MCS 23 ┤       /──────── ceiling converges ────────
       │      /
MCS 20 ┤─────/    ← Brief dips, fast recovery
       │
MCS 15 ┤     (no longer gets stuck here)
```

# Iterative Tuning — 5 Testbed Runs

| Run | Version | Fix | eMBB Avg | Loss | URLLC RTT |
|---|---|---|---|---|---|
| 1 | v4 pre-fix | — | 96.4 Mbps | 2.2% | 23 ms |
| 2 | v4 ceiling | Ceiling tracking | 101 Mbps | 0.88% | 25 ms |
| 3 | v4.1 | Relax faster | **81.5 Mbps ↓** | 0.52% | 24 ms |
| 4 | v4.2 | Dead-zone, floor | 89.5 Mbps | 0.65% | 23 ms |
| 5 | **v4.3** | **Cumul. drop cap** | **103 Mbps ✓** | 2.0% | 24 ms |

**Run 3 regression:** CQI=11 UEs hit floor=cap trap (MCS 8 for 400+ frames).

**v4.3 breakthrough:** Brief 1–2s dips instead of 5+ second traps. Peak 120 Mbps.

# v5 — Reinforcement Learning Extension

**Idea:** Replace static OLLA step sizes with a **Q-learning agent** that adapts per traffic class.

**State** (discretized):

- Average BLER bucket (5 levels)
- Average MCS bucket (4 levels)
- Pending bytes bucket (4 levels)

**Actions** (eMBB: 5, URLLC: 4):

- `(step_up, step_down)` tuples
- e.g., (0.25, 0.50) → conservative
- e.g., (0.75, 1.50) → aggressive

**Reward:**

- eMBB: throughput Δ – BLER penalty – backlog penalty
- URLLC: –BLER penalty – HOL delay penalty

**Runs at OLLA timescale** (~50 ms):

- Pure Lua Q-table → ~0.1 µs lookup
- ε-greedy with decay (0.15 → 0.02)
- No Python, no neural network
- Fits entirely in slot budget

# RL Agent Architecture

```
Every 50 ms (OLLA_PERIOD = 5 frames):

    ┌──────────────────────────────────────────────────────┐
    │  Aggregate per-class metrics from all UEs            │
    │  (avg BLER, avg MCS, total pending, avg HOL)         │
    └──────────────────────────────────────────────────────┘
                          │
                          ▼
    ┌──────────────────────────────────────────────────────┐
    │  Discretize → state_key = (bler, mcs, pending)       │
    │  Update Q(prev_state, prev_action) with reward       │
    │  Select new action (ε-greedy)                        │
    │  Set active_embb_up/down, active_urllc_up/down       │
    └──────────────────────────────────────────────────────┘
                          │
                          ▼
    ┌──────────────────────────────────────────────────────┐
    │  Fast Lua scheduler uses RL-tuned parameters         │
    │  for the next 50 ms of slot-level decisions          │
    └──────────────────────────────────────────────────────┘
```

Q-table has ~80 states × 5 actions = **400 entries** — trivially fits in LuaJIT memory.

# Key Design Decisions

| Decision | Our Choice | Why |
|---|---|---|
| Puncturing vs. reservation | Puncturing (Phase 2 first) | Better eMBB throughput when URLLC load is low |
| URLLC RB placement | Frequency-selective (best channel) | Better URLLC reliability, same RB cost |
| eMBB fairness | Compensated PF with $\rho_i$ tracking | Prevents starvation of punctured UEs |
| OLLA oscillation fix | Ceiling + dead-zone + cumul. cap | Converging damped oscillation vs. sawtooth |
| RL integration | Q-table at OLLA timescale | O(1) lookup, no Python, fits in slot budget |
| RL vs. heuristic | RL tunes parameters, heuristic decides | Best of both: structure + adaptability |

# Results Summary

## eMBB Performance (v4.3)

- **103 Mbps** average throughput
- **120 Mbps** peak
- 2.0% packet loss
- Brief dips (1–2s), fast recovery
- No prolonged MCS traps

## URLLC Performance

- **24 ms** average RTT (target: 50 ms)
- **0% packet loss** across all runs
- Consistent across 30+ ping samples
- Independent of eMBB load

## What Made the Difference

1. **MCS ceiling memory** — stopped the sawtooth
2. **CQI-derived floor** — prevented death spiral below channel capability
3. **Cumulative drop cap** — limited cascade depth
4. **Iterative testbed tuning** — 5 runs, each fixing a real observed failure mode

# File Structure

| File | Description |
|---|---|
| `pf_dl_slice_aware_v4.lua` | Production scheduler (v4.3) — best tested |
| `pf_dl_slice_aware_v4_csv.lua` | Same + CSV logging to `/mnt/shared/.../test-runs/` |
| `pf_dl_slice_aware_v5.lua` | RL extension with Q-table OLLA tuning |
| `pf_dl_simple.lua` | Baseline scheduler (NEU reference) |
| `entrypoint_oai.sh` | gNB entrypoint with scheduler path args |
| `ue_tester.py` | UE test controller |

## Running

```
export LUA_SCHED=/mnt/shared/open6g-ai-school-group6/pf_dl_slice_aware_v4.lua
# or for CSV logging:
export LUA_SCHED=/mnt/shared/open6g-ai-school-group6/pf_dl_slice_aware_v4_csv.lua
```

# Thank You

**Group 6** · Open6G AI School

OAI NR MAC DL Scheduler for eMBB/URLLC Coexistence

Source: `open6g-ai-school-group6/`